

Laboratory Notes

Week 5 Lab: SLAM and Navigation

1 Aim

This laboratory aims to explore the concepts of navigation and SLAM using the turtlebot3 model in Gazebo. It includes maze exploration via teleoperation, an automated exploration node and a wall-follower node. The laboratory will cover the following specific areas:

- Launching and examining several Gazebo Maze Worlds.
- Running a SLAM node to build maps of the Maze Worlds.
- Automated exploration of the mazes via a custom explorer node.
- Automated exploration of the mazes via a wall-follower node.
- Setting direct Navigation Goals in Nav2.

2 Background

Navigation is a core concept in autonomous systems. In particular, Simultaneous Localisation and Mapping (SLAM) has generally been considered as the key to making robots ‘truly autonomous’. This lab explores the general concepts of SLAM and Navigation in ROS2. We will examine and use some pre-prepared maze worlds and look at ways the turtlebot can automatically explore the mazes, while mapping the maze layouts. For this lab we will rely upon the navigation framework developed for ROS2, called Nav2. This package provides the foundational navigation tools, including global and local path planning, along with the necessary integration with SLAM packages (in our case SLAM Toolbox). The first two demonstrations in ECTE477 rely heavily upon maze exploration, navigation and mapping.

3 Procedure

We have provided a number of practice mazes, designed for testing and development. The ROS package containing these can be downloaded from GitHub (and built) via the following:

```
1 cd ~/ros2_ws/src
2 git clone https://github.com/UOW-SECTE-477/practice_mazes.git
3 cd ..
4 colcon build
```

If you navigate to the package you have just downloaded and open the launch directory, you will see that there are 9 worlds that you can run: three of these are simple mazes with very basic layouts, while six contain more complex maze layouts to explore. We will be adding more maze variations and other custom worlds throughout the session.

3.1 Running the Practice Mazes

Use the following command to launch the first practice maze:

```
1 ros2 launch practice_mazes maze_simple_0_world.launch.xml
```

After launching the first maze, replace the name of the launch file in the above command to have a look at some different mazes. Remember to shutdown each world before launching a new one (you cannot successfully run multiple Gazebo instances).

Now, launch *maze_simple_0_world.launch.xml* again. Try building a map with SLAM and teleoperation on this maze. Use the following commands and the keyboard to move the turtlebot and build a map in RVIZ:

```
1 ros2 launch nav2_bringup rviz_launch.py
2 ros2 launch nav2_bringup slam_launch.py
3 ros2 run turtlebot3_teleop teleop_keyboard
```

3.2 Initial Exploration

We will now look at building a map using an exploration algorithm, rather than manually moving the turtlebot via teleop. Download and build the following package from github:

```
1 cd ~/ros2_ws/src
2 git clone https://github.com/UOW-SECTE-477/ecte477_starter_python.git
3 cd ..
4 colcon build
```

Most exploration packages use a ‘frontier exploration’ algorithm. These algorithms rely on the published Occupancy Grid Map on the /map topic and search for frontiers (borders between explored and unexplored areas). They then issue movement commands to the ROS navigation system (Nav2 for ROS2). The one that we will be using is slightly different in that it does not search for frontiers but rather looks ahead, then left and right, for areas that are not blocked and then moves in those directions. If it is in a position where it finds multiple options for movement, it will move in an order of preference first and save the other option(s) to a private stack – the preference is always forward, then left and then right. Once it is blocked in the forward, left and right directions it will check if there are any unexplored options on the stack – if the stack is empty, it will finish exploring. This algorithm still uses the ROS navigation system for movements though, hence there

are a number of separate nodes that are required for this operation (rviz, slam, navigation and the exploration package itself).

First launch the gazebo world:

```
1 ros2 launch practice_mazes maze_simple_0_world.launch.xml
```

Once Gazebo has loaded, launch rviz, slam and the navigation nodes (in separate terminals):

```
1 ros2 launch nav2_bringup rviz_launch.py
2 ros2 launch nav2_bringup slam_launch.py
3 ros2 launch ecte477_navigation_launch.py use_sim_time:=true
```

Finally, we can start the exploration using:

```
1 ros2 run ecte477_starter_python maze_explorer
```

As soon as this node launches you should see the turtlebot begin moving throughout the maze, filling in any reachable gaps in the map and then stopping when there is nothing left to explore.

Note: if you get an error regarding the *tf_transformations* module, this means that the module is not installed on your system. You can install it by running the following command (please see your lab demonstrator if you require help with this).

```
1 sudo apt install ros-jazzy-tf-transformations
```

3.3 Building a Launch File for Exploration

Launching four different nodes over the command line can be burdensome, so now we will build our own launch file that will take care of all of this. In your own package, make a directory called *launch* (if it does not already exist) then (in VS Code) make a new file in your launch directory called *start_exploring.launch.xml*

Add the following lines of code to your new launch file (don't forget to re-build your package):

```
1 <launch>
2   <set_parameter name="use_sim_time" value="true" />
3
4   <!-- SLAM -->
5   <include file="$(find-pkg-share nav2_bringup)/launch/slam_launch.py"/>
6
7   <!-- Navigation -->
8   <include file="$(find-pkg-share ecte477)/launch/navigation_launch.py">
9     <arg name="params_file" value="$(find-pkg-share ecte477)/params/nav2_params.yaml"/>
10  </include>
11
12  <!-- RVIZ -->
13  <node name="rviz2" pkg="rviz2" exec="rviz2" args="-d $(find-pkg-share ecte477_starter_python)/config/my_rviz.rviz"/>
14
15  <!-- Explore -->
16  <node name="explore_tb3" pkg="ecte477_starter_python" exec="maze_explorer" output="screen"/>
17 </launch>
```

Now, after opening a maze, you can launch the navigation stack and exploration together (substitute the name of your package for *my_package*):

```
1 ros2 launch practice_mazes maze_simple_0_world.launch.xml
2 ros2 launch my_package start_exploring.launch.xml
```

All of the nodes required for exploration should now start from the one launch command. Note also that, in our launch file, we provided rviz with a custom configuration rather than using the standard nav2_bringup configuration.

3.4 Wall Following

Inside the ecte477_starter_python package (that you have already built), you will also find a wall following node. This node will navigate a maze via following walls, without using the full Nav2 stack. It does not keep track of where it has been, so never knows if the full maze has been explored. Hence, it must be started and stopped manually; however, it can also be used with slam to build a map of the maze.

To start the wall follower, we need to launch both the Command Server and Wall Follower nodes. First, load a maze of your choosing. Next, run the following commands (in separate terminals):

```
1 ros2 launch nav2_bringup rviz_launch.py
2 ros2 launch nav2_bringup slam_launch.py
3 ros2 run ecte477_starter_python command_server
4 ros2 run ecte477_starter_python wall_follower
```

Now, we can start and stop the wall follower by publishing the following messages to the cmd topic over the terminal (when required):

```
1 ros2 topic pub -1 /cmd std_msgs/msg/String "{data:'start'}"
2 ros2 topic pub -1 /cmd std_msgs/msg/String "{data:'stop'}"
```

After running the wall_follower for the first time, see if you can again write a new launch file to open all of the required nodes for wall following from a single launch command. Refer to the launch file you wrote in Section 3.3 as a guide. Re-build your package and run the wall follower again from your new launch file.

The operation of the wall follower is described in Figure 1. The node takes a message of type *sensor_msgs/LaserScan* which contains 360 distance measurements taken from the laser scanner in one degree increments around the Turtlebot. These distances are converted into 3D points relative to the robot centre and then tested to see if they lie within either of two search regions, shown in Figures 1a and ??b. For the left region, the node finds the location on the x-axis of the forward-most point, $x_{s,max}$. For the front region, the node finds the location on the x-axis of the closest wall point, $x_{f,min}$. If $x_{s,max}$ is behind the front of the robot, this means the Turtlebot is driving past the end of a left turn and should begin to slow down and turn left. If $x_{f,min}$ is closer than some maximum, then the Turtlebot is approaching a wall that should be kept on the left, so it should begin to slow down and turn right. The wall follower combines these two decisions to determine the Turtlebot's drive and turn speed.

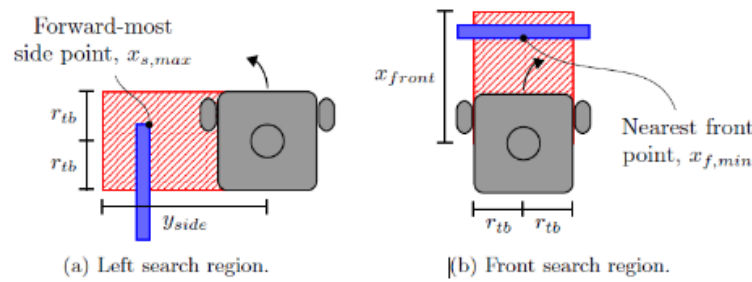


Figure 1: Wall Follower Operation.

3.5 Nav2

Throughout the course we will be using Nav2, which is a navigation framework developed for ROS2. This replaces the ROS Navigation Stack that was used in ROS1. We generally access Nav2 through the `nav2_bringup` package. A “bringup” package is one where launch files have been prepared to bring-up all important nodes for a specific system or application. They are generally designed so that you can mirror them and modify appropriately for your specific needs – notice that the `navigation_launch.py` file we are using is in the `ecte477` package, this is because we have made such modifications to this particular launch file. In our case we can start all of the important Nav2 navigation nodes with two launch files:

```
1 ros2 launch ecte477 navigation_launch.py
2 ros2 launch nav2_bringup slam_launch.py
```

The first launch file required is the `navigation_launch.py` file. This brings up nodes for path planning (both global and local costmaps are maintained), issuing command velocities based on the generated path, and setting navigation goals – along with setting (and following) sequential waypoints towards a goal. For these nodes to function correctly a map and method of localisation also need to be provided. The nodes responsible for the map and localisation are brought up by our second launch file.

There are two different ways to provide the map and localisation within the `nav2_bringup` package. If you do not have an existing map, the `slam_launch.py` file (shown above) can be used. The slam launch file uses `slam_toolbox` to build the map as the robot runs and provides this information, along with localisation, to the navigation system. If you have an existing map of your robot’s workspace you should instead use the `localization_launch.py` file (shown below). This provides map server and localisation nodes for the navigation system. Localisation is provided by an adaptive Monte Carlo (AMCL) algorithm, which uses a particle filter approach to localisation.

```
1 ros2 launch nav2_bringup localization_launch.py
```

You will note that both the navigation and slam launch files were included (via `<include>` tags) in our `start_exploring.launch.xml` file that we wrote earlier in this lab. This is because the `maze_explorer` node relies on the Nav2 navigation system. It operates by finding navigation goals within the maze and sending them to the navigation system. You should have also noticed that we have been setting a parameter `“use_sim_time”` to `“true”` every time we have called the navigation launch file. This is to tell Nav2 that we are using a Gazebo

simulation and not a real robot – hence, all robot state data that is published is based on the simulation clock from Gazebo.

3.6 Setting a Navigation Goal

We will now interact with the navigation system, by setting our own navigation goals. To test this out we will start both the slam and navigation launch files discussed. Note that we also need to be running a Gazebo simulation and RVIZ. Hence:

```
1 ros2 launch practice_mazes maze_simple_0_world.launch.xml
2 ros2 launch nav2_bringup rviz_launch.py
3 ros2 launch nav2_bringup slam_launch.py
4 ros2 launch ecte477 navigation_launch.py use_sim_time:=true
```

The simplest way to set a navigation goal is to publish to the `/goal_pose` topic, which takes a *geometry_msgs/msg/PoseStamped* message. A *geometry_msgs/msg/PoseStamped* contains a *std_msgs/msg/Header* and a *geometry_msgs/msg/Pose*, with the Pose message itself containing a *geometry_msgs/msg/Point* and a *geometry_msgs/msg/Quaternion*. You can check these for yourself using “ros2 interface show” on each message type.

To move the Turtlebot to point [2.0, 0.0, 0.0] with orientation [0.0, 0.0, 0.0, 1.0] we can publish the following message (note that this is all one line):

```
1 ros2 topic pub -1 /goal_pose geometry_msgs/msg/PoseStamped "{header: {frame_id: 'map'}, pose: {position: {x: 2.0, y: 0.0, z: 0.0}, orientation: {w: 1.0}}}"
```

If you run this in the terminal, you should see the turtlebot move forward by two metres. Remember that the goal we published is in the global frame and isn’t relative to the robot’s current location. The robot just happened to have an initial position of [0.0; 0.0; 0.0].

An equivalent message created in Python would look something like the following:

```
1 from geometry_msgs.msg import PoseStamped
2
3 goal_msg = PoseStamped()
4 goal_msg.header.frame_id = 'map'
5 goal_msg.pose.position.x = 2.0
6 goal_msg.pose.position.y = 0.0
7 goal_msg.pose.position.z = 0.0
8 goal_msg.pose.orientation.x = 0.0
9 goal_msg.pose.orientation.y = 0.0
10 goal_msg.pose.orientation.z = 0.0
11 goal_msg.pose.orientation.w = 1.0
```

Practice navigating the Turtlebot around by publishing goal poses.

Some other ways of setting navigation goals:

- You can also set navigation goals for Nav2 directly in RVIZ. With the same simulation running, including all of the navigation nodes, try navigating the robot from RVIZ. You can do this by selecting

“Nav2Goal” in the top bar – once selected you can click on the map in RVIZ and drag a big green arrow (see the example in Figure 2). The goal’s position is set as where you first clicked on the map and the orientation is set by the direction of the arrow you dragged.

- Using RVIZ you can also set waypoints for your robot to move through. In the bottom left corner of your RVIZ screen you should see a button called “Waypoint/Nav Through Poses Mode”. Press this now and set some waypoints (you set each individual waypoint using the same “Nav2Goal” selection that you used for a setting a single goal). Once you have a few waypoints, press the “Start Waypoint Following” button that is now showing in the bottom left corner of RVIZ. You should see your Turtlebot navigate through each of the waypoints that you set (in order).
- The principle way to set navigation goals programmatically is to use the “NavigateToPose” action server provided in Nav2. The action server allows you to receive feedback as the navigation system processes the goal, executes the path planning and sends commands to the robot. Some example code of using this action server is provided after Figure 2. This sample code is taken from the maze_explorer node.

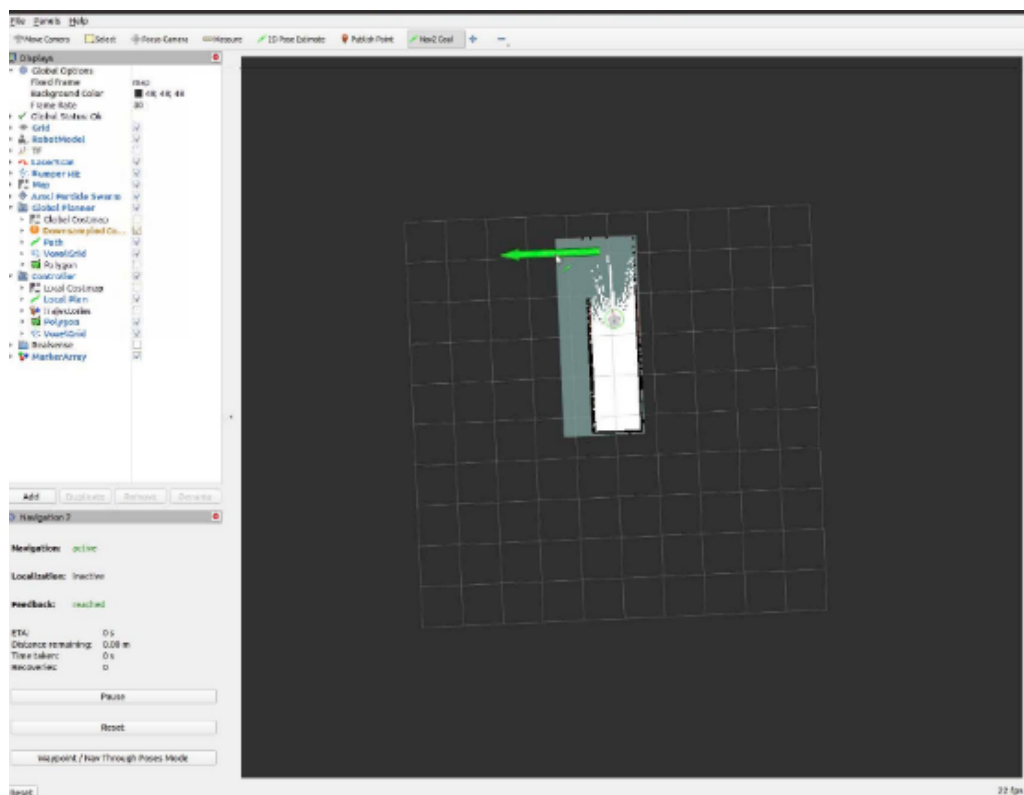


Figure 2: Navigation via RVIZ.

Navigation sample code:

```

1  def goGoal(self, goal):
2      self.goal_reached = False
3      action_request = NavigateToPose.Goal()
4      action_request.pose = goal

```

```
5
6     self.move_action.wait_for_server()
7     self.get_logger().info('Sending goal request .....')
8     send_goal_future = self.move_action.send_goal_async(action_request)
9
10    try:
11        rclpy.spin_until_future_complete(self, send_goal_future)
12        self.goal_handle = send_goal_future.result()
13    except Exception as e:
14        self.get_logger().info('Service call failed %r' % (e,))
15
16    if not self.goal_handle.accepted:
17        self.get_logger().info('Goal rejected')
18    else:
19        self.get_logger().info('Goal accepted')
20
21    get_result_future = self.goal_handle.get_result_async()
22
23    self.get_logger().info("Waiting for 'Navigate to Pose' action to complete...")
24    try:
25        rclpy.spin_until_future_complete(self, get_result_future)
26        self.get_logger().info('goal reached')
27        time.sleep(5)
28        self.goal_reached = True
29        return
30    except Exception as e:
31        self.get_logger().info('Service call failed %r' % (e,))
```

4 Summary

In today's laboratory we have explored the concepts of navigation and SLAM in ROS. We first loaded in some custom practice mazes developed for the Turtlebot3 and then mapped one of these mazes using SLAM and teleoperation. We then explored the maze using both a custom maze explorer node and a custom wall following node. We looked at building launch files in order to launch all of the required navigation and mapping nodes via a single launch command. Finally, we looked at how navigation goals can be set in Nav2. Navigation and mapping are core concepts in autonomous systems and will be used during the upcoming demonstration tasks for ECTE477.